

STUDENT SCHOLARSHIP PORTAL

A system for applying and vetting candidates based on GPA. Automates ranking using specific database criteria.

Submitted by:

CARTONEROS, Beomarc Andrew D.

CARVAJAL, Christian Ezekiel L.

ESCABAL, Axeil Achia M.

GO, Marco Enrico S.

MOSOMOS, Jairus John U.

Submitted to:

Prof. Roberto Malitao

Date Submitted:

April 15, 2026

Tech Stack Justification

1. Frontend Architecture

Technology Stack

HTML5, CSS3, and ES6+ JavaScript

Core Components

index.html, login.html, admin-portal.html, student-portal.html, style.css

JavaScript modules

- js/auth.js
- js/admin.js
- js/app.js

Overview

The frontend follows a Vanilla Web Stack approach, meaning it does not rely on frameworks such as React, Angular, or Vue.js. This keeps the system direct and efficient.

Key Design Principles

- **Zero-Build Pipeline**

The system removes the need for tools like Webpack, Vite, or Babel. There is no compilation step, which simplifies development and avoids dependency conflicts. It also lowers the risk of vulnerabilities from third-party packages.

- **Separation of Concerns**

The JavaScript structure is divided into clear responsibilities

- auth.js handles authentication logic such as login and logout
- admin.js isolates administrative actions and prevents overlap with student features
- app.js manages shared logic and reusable utilities

This separation avoids mixing responsibilities and improves maintainability.

- **Performance Efficiency**

Static files are served directly from the public directory. Since there is no large JavaScript bundle to process, the application achieves faster First Contentful Paint and quicker user interaction readiness.

- **Accessibility and Simplicity**

Using standard web technologies makes the project easier to understand and extend, especially for new developers.

2. Backend Framework

Technology Stack

Node.js with Express (v5.x)

Core Components

server.js, package.json

Overview

The backend is built using a JavaScript runtime and a minimal web framework to handle API requests and server logic.

Key Design Principles

- **Unified Language Across Stack**

JavaScript is used on both client and server sides. This reduces complexity and allows faster development since the same language is applied everywhere.

- **Modern Express Features**

Express version 5 supports native Promise handling. Errors from asynchronous operations are automatically passed to a global handler, reducing repetitive error management code.

- **Flexible Architecture**

The backend can serve static frontend files while also handling dynamic API routes in the same environment.

3. Software Design Pattern

Architecture Style

Modular MVC-like Structure

Core Components

applicationRoutes.js, applicationController.js

Overview

The system follows a structured pattern that separates responsibilities between routing and business logic.

Key Design Principles

- **Routing Layer**
Routes define API endpoints and map requests to specific controller functions. This keeps the server entry file clean and easy to read.
 - **Controller Layer**
Controllers contain the main business logic. They process incoming requests, interact with the database, and return responses.
 - **Loose Coupling**
Because routing and logic are separated, changes in one layer do not heavily affect the other. This makes the system easier to scale or modify in the future.
-

4. Database Layer

Technology Stack

MySQL with mysql2 (v^3.21.1)

Core Components

db.js, setup-db.js

Overview

The system uses a relational database to manage structured data such as student and administrative records.

Key Design Principles

- **Relational Integrity**
MySQL enforces structured schemas and relationships between entities. This ensures that records remain consistent and valid.
- **ACID Compliance**
Transactions are reliable and safe, which is important when handling sensitive or critical data.
- **High Performance Queries**
Instead of using heavy ORMs, the system uses direct SQL queries through mysql2. This reduces overhead and gives better control over performance.

- **Asynchronous Operations**

The mysql2 driver supports Promise-based queries, allowing smooth integration with async and await patterns.

- **Automated Database Setup**

The setup-db.js script allows automatic creation of tables and initial data. This reduces manual setup and improves deployment efficiency.

5. Security and Environment Management

Technology Stack

dotenv (v^17.4.1)

Overview

The system manages sensitive data using environment variables instead of hardcoding them in the source code.

Key Design Principles

- **Secure Configuration Handling**

Credentials such as database passwords and API keys are stored in a .env file, keeping them hidden from public repositories.

- **Environment Flexibility**

Different configurations can be used for development, testing, and production without changing the code.

- **Standard Practice Alignment**

This approach follows widely accepted secure development practices for modern applications.

6. AI Integration

Technology Used

Gemini 3.1 Pro (Preview)

Overview

An AI component is integrated to support advanced system capabilities such as intelligent processing or automation features.

Key Value

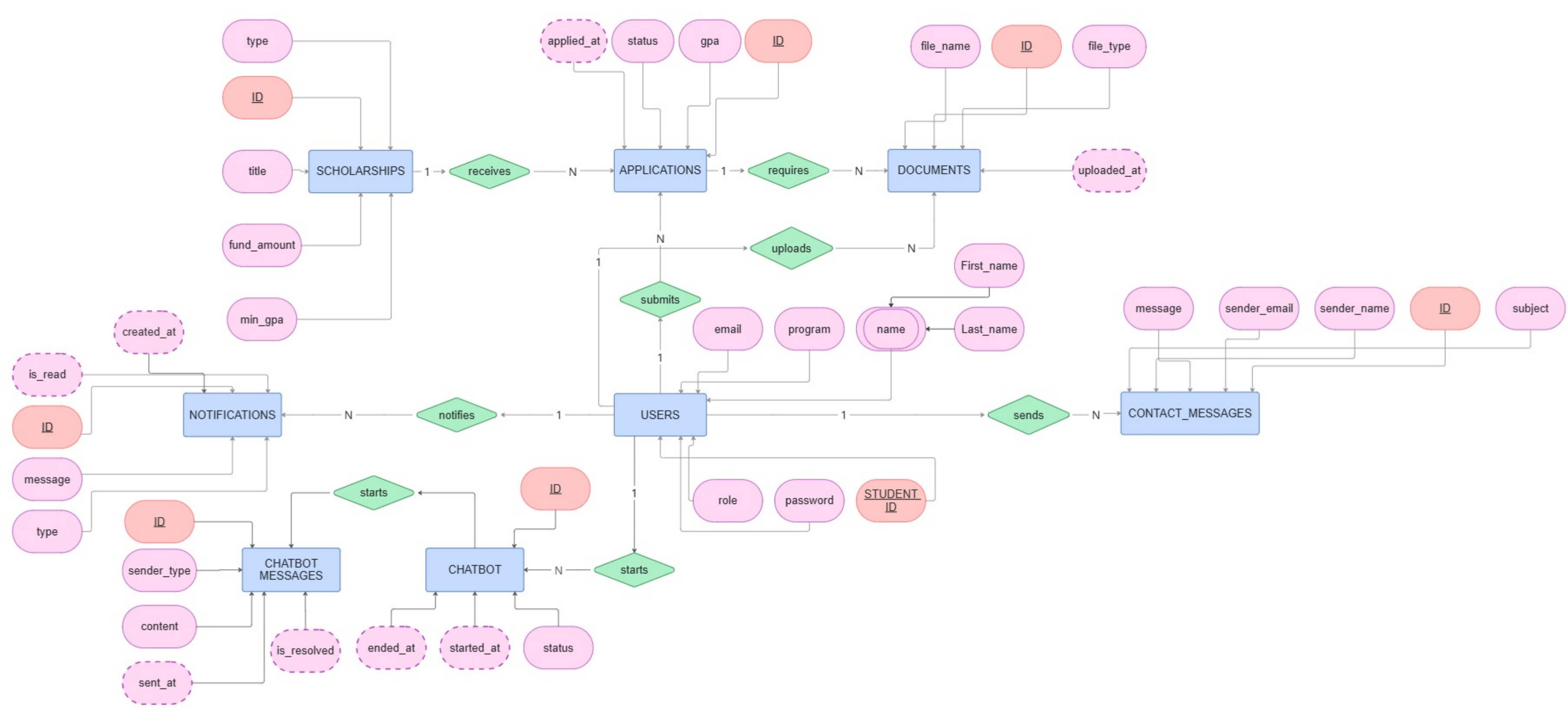
- Enhances system functionality beyond traditional logic-based operations
 - Allows future expansion into smarter features without major redesign
-

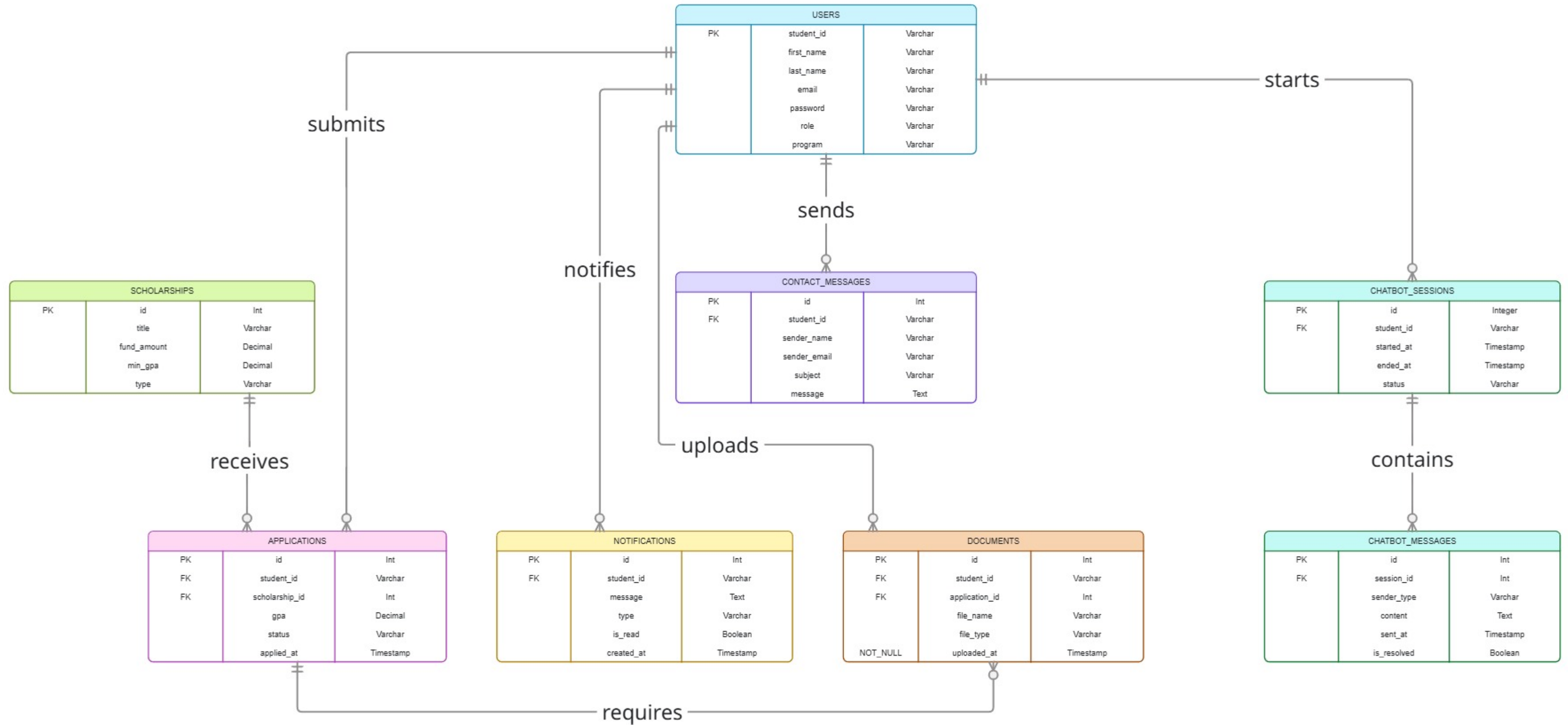
7. Overall Architecture Summary

The system follows a **monolithic client-server architecture** where the frontend, backend, and database are tightly integrated within a single deployment.

Value Proposition

- **Simplicity**
Minimal dependencies and no unnecessary layers make the system easy to understand and maintain
- **Performance**
Direct serving of static files and optimized backend logic ensures fast execution
- **Maintainability**
Clear separation of concerns and modular structure allow easier updates and scaling
- **Practical Engineering Approach**
The design avoids over-engineering by not using heavy frameworks or ORMs, focusing instead on efficiency and clarity





Data Dictionary

School Scholarship Portal

8 tables · Constraint legend below

[PK] Primary key **[FK]** Foreign key **[Not null]** Required field **[Auto inc]** Auto increment **[Unique]** Unique constraint **[Nullable]** Optional field

Table 1 — Students

Field name	Data type	Length	Constraints	Default	Description
student_id	VARCHAR	50	[PK] [Not null]	—	Unique student identifier
first_name	VARCHAR	50	[Not null]	—	Given name
last_name	VARCHAR	50	[Not null]	—	Family name
name	VARCHAR	255	[Nullable]	NULL	Full name
program	VARCHAR	150	[Nullable]	NULL	Academic program
email	VARCHAR	100	[Unique] [Not null]	—	Login email
password_hash	VARCHAR	255	[Not null]	—	Encrypted password
major	VARCHAR	100	[Nullable]	NULL	Field of study
year_level	VARCHAR	50	[Not null]	—	Academic level
gpa	DECIMAL	3,2	[Not null]	—	Grade Point Average
created_at	TIMESTAMP	—	[Not null]	CURRENT_TIMESTAMP	Record timestamp

Table 2 — Scholarships

Field name	Data type	Length	Constraints	Default	Description
id	INT	—	[PK] [Auto inc]	—	Scholarship ID
title	VARCHAR	255	[Not null]	—	Scholarship title
name	VARCHAR	100	[Nullable]	NULL	Alternate name
description	TEXT	—	[Nullable]	NULL	Full description
fund_amount	DECIMAL	10,2	[Not null]	—	Award amount
min_gpa	DECIMAL	3,2	[Not null]	—	Required GPA
deadline	DATE	—	[Not null]	—	Application deadline
type	VARCHAR	50	[Not null]	—	Scholarship type
created_at	TIMESTAMP	—	[Not null]	CURRENT_TIMESTAMP	Record timestamp

Table 3 — Applications

Field name	Data type	Length	Constraints	Default	Description
id	INT	—	[PK] [Auto inc]	—	Application ID
student_id	VARCHAR	50	[FK] [Not null]	—	Ref: students
scholarship_id	INT	—	[FK] [Not null]	—	Ref: scholarships
status	ENUM	—	[Not null]	Pending	Application status
score	INT	—	[Not null]	0	Evaluation score
transcript_path	VARCHAR	255	[Nullable]	NULL	Transcript file path
id_path	VARCHAR	255	[Nullable]	NULL	ID file path
applied_at	TIMESTAMP	—	[Not null]	CURRENT_TIMESTAMP	Submission timestamp

Table 4 — Documents

Field name	Data type	Length	Constraints	Default	Description
id	INT	—	[PK] [Auto inc]	—	Document ID
student_id	VARCHAR	50	[FK] [Not null]	—	Ref: students
application_id	INT	—	[FK] [Not null]	—	Ref: applications
file_name	VARCHAR	255	[Not null]	—	File name
file_type	VARCHAR	50	[Not null]	—	File type/extension
uploaded_at	TIMESTAMP	—	[Not null]	—	Upload timestamp

Table 5 — Notifications

Field name	Data type	Length	Constraints	Default	Description
id	INT	—	[PK] [Auto inc]	—	Notification ID
student_id	VARCHAR	50	[FK] [Not null]	—	Ref: students
message	TEXT	—	[Not null]	—	Notification content
type	VARCHAR	50	[Not null]	—	Notification type
is_read	BOOLEAN	—	[Not null]	—	Read status
created_at	TIMESTAMP	—	[Not null]	—	Record timestamp

Table 6 — Contact Messages

Field name	Data type	Length	Constraints	Default	Description
id	INT	—	[PK] [Auto inc]	—	Message ID
student_id	VARCHAR	50	[FK] [Nullable]	NULL	Ref: students (optional)
sender_name	VARCHAR	100	[Not null]	—	Sender name
sender_email	VARCHAR	100	[Not null]	—	Sender email
subject	VARCHAR	255	[Not null]	—	Message subject
message	TEXT	—	[Not null]	—	Message body

Table 7 — Chatbot Sessions

Field name	Data type	Length	Constraints	Default	Description
id	INT	—	[PK] [Auto inc]	—	Session ID
student_id	VARCHAR	50	[FK] [Nullable]	NULL	Ref: students (optional)
started_at	TIMESTAMP	—	[Not null]	—	Session start time
ended_at	TIMESTAMP	—	[Nullable]	NULL	Session end time
status	VARCHAR	50	[Nullable]	NULL	Session status

Table 8 — Chatbot Messages

Field name	Data type	Length	Constraints	Default	Description
id	INT	—	[PK] [Auto inc]	—	Message ID
session_id	INT	—	[FK] [Not null]	—	Ref: chatbot_sessions
sender_type	VARCHAR	50	[Not null]	—	bot or user
content	TEXT	—	[Not null]	—	Message content
sent_at	TIMESTAMP	—	[Not null]	—	Send timestamp
is_resolved	BOOLEAN	—	[Not null]	—	Resolution status

Initial SQL Script

-- School Scholarship Portal (MySQL 8.0+) - SINGLE UNIFIED SCRIPT

-- Includes:

-- 1) Database + Tables (DDL) with Audit & Notification systems

-- 2) Admin, Faculty, and Student separate accounts + role enforcement

-- 3) Stored Procedures (transaction logic + concurrency fixes)

-- 4) Yearly scholarships + GPA maintenance + per-semester installments

-- 5) Probation logic (fail once => probation, fail again => terminated)

CREATE DATABASE *IF NOT EXISTS* scholarship_portal;

USE scholarship_portal;

-- =====

-- 1) CORE TABLES (Roles & Users)

-- =====

CREATE TABLE *IF NOT EXISTS* users (

 user_id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,

 email VARCHAR(255) NOT NULL,

 password_hash VARCHAR(255) NOT NULL,

 role ENUM('student','faculty','admin') NOT NULL,

 is_active TINYINT(1) NOT NULL DEFAULT 1,

 created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,

 PRIMARY KEY (user_id),

 UNIQUE KEY uk_users_email (email)

) ENGINE=InnoDB;

```
CREATE TABLE IF NOT EXISTS students (  
    student_id    BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,  
    user_id       BIGINT UNSIGNED NULL,  
    student_number VARCHAR(50) NOT NULL,  
    first_name    VARCHAR(100) NOT NULL,  
    last_name     VARCHAR(100) NOT NULL,  
    gpa           DECIMAL(5,2) NULL, -- OPTIMIZED: Flexible GPA  
    major         VARCHAR(150) NULL,  
    created_at    TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    PRIMARY KEY (student_id),  
    UNIQUE KEY uk_students_student_number (student_number),  
    UNIQUE KEY uk_students_user_id (user_id),  
    CONSTRAINT fk_students_user  
        FOREIGN KEY (user_id) REFERENCES users(user_id)  
        ON DELETE CASCADE  
) ENGINE=InnoDB;
```

```
CREATE TABLE IF NOT EXISTS staff (  
    staff_id    BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,  
    user_id     BIGINT UNSIGNED NOT NULL,  
    faculty_number VARCHAR(50) NOT NULL,  
    first_name  VARCHAR(100) NOT NULL,  
    last_name   VARCHAR(100) NOT NULL,  
    department  VARCHAR(150) NULL,  
    created_at  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    PRIMARY KEY (staff_id),
```

```

    UNIQUE KEY uk_staff_faculty_number (faculty_number),
    UNIQUE KEY uk_staff_user_id (user_id),
    CONSTRAINT fk_staff_user
        FOREIGN KEY (user_id) REFERENCES users(user_id)
        ON DELETE CASCADE
) ENGINE=InnoDB;

CREATE TABLE IF NOT EXISTS admins (
    admin_id    BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    user_id     BIGINT UNSIGNED NOT NULL,
    admin_number VARCHAR(50) NOT NULL,
    first_name  VARCHAR(100) NOT NULL,
    last_name   VARCHAR(100) NOT NULL,
    created_at  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (admin_id),
    UNIQUE KEY uk_admins_admin_number (admin_number),
    UNIQUE KEY uk_admins_user_id (user_id),
    CONSTRAINT fk_admins_user
        FOREIGN KEY (user_id) REFERENCES users(user_id)
        ON DELETE CASCADE
) ENGINE=InnoDB;

```

```

-- =====
-- 2) ACADEMIC CALENDAR
-- =====

```

```
CREATE TABLE IF NOT EXISTS term (  
    term_id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,  
    code VARCHAR(20) NOT NULL,  
    name VARCHAR(100) NOT NULL,  
    start_date DATE NOT NULL,  
    end_date DATE NOT NULL,  
    PRIMARY KEY (term_id),  
    UNIQUE KEY uk_term_code (code)  
) ENGINE=InnoDB;
```

```
CREATE TABLE IF NOT EXISTS academic_years (  
    academic_year_id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,  
    code VARCHAR(20) NOT NULL,  
    name VARCHAR(50) NOT NULL,  
    start_date DATE NOT NULL,  
    end_date DATE NOT NULL,  
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    PRIMARY KEY (academic_year_id),  
    UNIQUE KEY uk_academic_years_code (code)  
) ENGINE=InnoDB;
```

```
CREATE TABLE IF NOT EXISTS evaluation_periods (  
    evaluation_period_id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,  
    academic_year_id BIGINT UNSIGNED NOT NULL,  
    period_code ENUM('SEM1','SEM2') NOT NULL,  
    start_date DATE NOT NULL,
```



```

end_date      DATE NOT NULL,
created_at    TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
PRIMARY KEY (evaluation_period_id),
UNIQUE KEY uk_eval_period (academic_year_id, period_code),
CONSTRAINT fk_eval_period_year
    FOREIGN KEY (academic_year_id) REFERENCES academic_years(academic_year_id)
    ON DELETE RESTRICT
) ENGINE=InnoDB;

```

```

-- =====
-- 3) SCHOLARSHIPS & APPLICATIONS
-- =====

```

```

CREATE TABLE IF NOT EXISTS scholarships (
    scholarship_id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    term_id        BIGINT UNSIGNED NOT NULL,
    academic_year_id BIGINT UNSIGNED NULL,
    name           VARCHAR(200) NOT NULL,
    description     TEXT NULL,
    amount         DECIMAL(12,2) NOT NULL,
    currency       CHAR(3) NOT NULL DEFAULT 'USD',
    max_awards     INT NOT NULL,
    maintain_min_gpa DECIMAL(5,2) NOT NULL DEFAULT 0.00, -- OPTIMIZED
    disbursement_plan ENUM('per_semester') NOT NULL DEFAULT 'per_semester',
    application_open DATETIME NOT NULL,
    application_close DATETIME NOT NULL,

```

```

is_active      TINYINT(1) NOT NULL DEFAULT 1,
created_at     TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
PRIMARY KEY (scholarship_id),
KEY idx_scholarships_term (term_id),
KEY idx_scholarships_year (academic_year_id),
KEY idx_scholarships_window (application_open, application_close),
CONSTRAINT fk_scholarships_term FOREIGN KEY (term_id) REFERENCES term(term_id)
ON DELETE RESTRICT,
CONSTRAINT fk_scholarships_academic_year FOREIGN KEY (academic_year_id)
REFERENCES academic_years(academic_year_id) ON DELETE RESTRICT
) ENGINE=InnoDB;

```

```

CREATE TABLE IF NOT EXISTS applications (
  application_id  BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  scholarship_id  BIGINT UNSIGNED NOT NULL,
  term_id        BIGINT UNSIGNED NOT NULL,
  student_id     BIGINT UNSIGNED NOT NULL,
  status         ENUM('draft','submitted','under_review','approved','rejected','withdrawn') NOT
  NULL DEFAULT 'draft',
  personal_statement TEXT NULL,
  submitted_at   DATETIME NULL,
  created_at     TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at     TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
  CURRENT_TIMESTAMP,
  PRIMARY KEY (application_id),
  UNIQUE KEY uk_app_unique (student_id, scholarship_id, term_id),
  KEY idx_app_student (student_id),

```

```
KEY idx_app_scholarship (scholarship_id),  
  
KEY idx_app_status (status),  
  
CONSTRAINT fk_app_scholarship FOREIGN KEY (scholarship_id) REFERENCES  
scholarships(scholarship_id) ON DELETE RESTRICT,  
  
CONSTRAINT fk_app_term FOREIGN KEY (term_id) REFERENCES term(term_id) ON  
DELETE RESTRICT,  
  
CONSTRAINT fk_app_student FOREIGN KEY (student_id) REFERENCES  
students(student_id) ON DELETE RESTRICT  
  
) ENGINE=InnoDB;
```

```
CREATE TABLE IF NOT EXISTS document_types (  
  
document_type_id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,  
  
code          VARCHAR(50) NOT NULL,  
  
name          VARCHAR(150) NOT NULL,  
  
is_required   TINYINT(1) NOT NULL DEFAULT 0,  
  
PRIMARY KEY (document_type_id),  
  
UNIQUE KEY uk_doc_types_code (code)  
  
) ENGINE=InnoDB;
```

```
CREATE TABLE IF NOT EXISTS application_documents (  
  
application_document_id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,  
  
application_id  BIGINT UNSIGNED NOT NULL,  
  
document_type_id BIGINT UNSIGNED NOT NULL,  
  
file_name      VARCHAR(255) NOT NULL,  
  
file_url       VARCHAR(500) NOT NULL,  
  
uploaded_at    TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  
PRIMARY KEY (application_document_id),
```

```

    UNIQUE KEY uk_app_doc_once (application_id, document_type_id),

    CONSTRAINT fk_app_docs_application FOREIGN KEY (application_id) REFERENCES
    applications(application_id) ON DELETE CASCADE,

    CONSTRAINT fk_app_docs_doc_type FOREIGN KEY (document_type_id) REFERENCES
    document_types(document_type_id) ON DELETE RESTRICT

) ENGINE=InnoDB;

```

```

CREATE TABLE IF NOT EXISTS reviews (

    review_id    BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,

    application_id BIGINT UNSIGNED NOT NULL,

    staff_id     BIGINT UNSIGNED NOT NULL,

    score        DECIMAL(5,2) NULL,

    decision     ENUM('needs_info','recommend_approve','recommend_reject') NOT NULL,

    comments     TEXT NULL,

    created_at   TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,

    PRIMARY KEY (review_id),

    CONSTRAINT fk_reviews_app FOREIGN KEY (application_id) REFERENCES
    applications(application_id) ON DELETE CASCADE,

    CONSTRAINT fk_reviews_staff FOREIGN KEY (staff_id) REFERENCES staff(staff_id) ON
    DELETE RESTRICT

) ENGINE=InnoDB;

```

```

-- =====
-- 4) AWARDS & DISBURSEMENTS
-- =====

```

```

CREATE TABLE IF NOT EXISTS awards (

```

```

award_id    BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
application_id BIGINT UNSIGNED NOT NULL,
scholarship_id BIGINT UNSIGNED NOT NULL,
student_id  BIGINT UNSIGNED NOT NULL,
term_id     BIGINT UNSIGNED NOT NULL,
amount_awarded DECIMAL(12,2) NOT NULL,
status      ENUM('pending','authorized','paid','cancelled') NOT NULL DEFAULT 'pending',
standing    ENUM('active','probation','terminated') NOT NULL DEFAULT 'active',
awarded_at  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
updated_at  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP, -- OPTIMIZED

PRIMARY KEY (award_id),
UNIQUE KEY uk_awards_application (application_id),
KEY idx_awards_sch_term (scholarship_id, term_id),

CONSTRAINT fk_awards_application FOREIGN KEY (application_id) REFERENCES
applications(application_id) ON DELETE RESTRICT,

CONSTRAINT fk_awards_scholarship FOREIGN KEY (scholarship_id) REFERENCES
scholarships(scholarship_id) ON DELETE RESTRICT,

CONSTRAINT fk_awards_student FOREIGN KEY (student_id) REFERENCES
students(student_id) ON DELETE RESTRICT,

CONSTRAINT fk_awards_term FOREIGN KEY (term_id) REFERENCES term(term_id) ON
DELETE RESTRICT

) ENGINE=InnoDB;

```

```

CREATE TABLE IF NOT EXISTS fund_sources (
fund_source_id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
code          VARCHAR(50) NOT NULL,

```

```
name      VARCHAR(150) NOT NULL,  
PRIMARY KEY (fund_source_id),  
UNIQUE KEY uk_fund_sources_code (code)  
) ENGINE=InnoDB;
```

```
CREATE TABLE IF NOT EXISTS disbursements (  
    disbursement_id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,  
    award_id      BIGINT UNSIGNED NOT NULL,  
    fund_source_id BIGINT UNSIGNED NOT NULL,  
    evaluation_period_id BIGINT UNSIGNED NULL,  
    amount        DECIMAL(12,2) NOT NULL,  
    status        ENUM('scheduled','paid','failed','cancelled') NOT NULL DEFAULT 'scheduled',  
    paid_at       DATETIME NULL,  
    updated_at    TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE  
CURRENT_TIMESTAMP, -- OPTIMIZED  
    PRIMARY KEY (disbursement_id),  
    CONSTRAINT fk_disb_award FOREIGN KEY (award_id) REFERENCES awards(award_id)  
ON DELETE CASCADE,  
    CONSTRAINT fk_disb_fund FOREIGN KEY (fund_source_id) REFERENCES  
fund_sources(fund_source_id) ON DELETE RESTRICT,  
    CONSTRAINT fk_disb_eval_period FOREIGN KEY (evaluation_period_id) REFERENCES  
evaluation_periods(evaluation_period_id) ON DELETE RESTRICT  
) ENGINE=InnoDB;
```

```
CREATE TABLE IF NOT EXISTS scholar_evaluations (  
    scholar_evaluation_id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,  
    award_id              BIGINT UNSIGNED NOT NULL,
```

```

evaluation_period_id BIGINT UNSIGNED NOT NULL,
evaluated_by_staff_id BIGINT UNSIGNED NOT NULL,
gpa_achieved      DECIMAL(5,2) NOT NULL, -- OPTIMIZED
decision          ENUM('pass','fail') NOT NULL,
notes            TEXT NULL,
evaluated_at      TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
PRIMARY KEY (scholar_evaluation_id),
UNIQUE KEY uk_eval_award_period (award_id, evaluation_period_id),
CONSTRAINT fk_eval_award FOREIGN KEY (award_id) REFERENCES awards(award_id)
ON DELETE CASCADE,

CONSTRAINT fk_eval_period FOREIGN KEY (evaluation_period_id) REFERENCES
evaluation_periods(evaluation_period_id) ON DELETE RESTRICT,

CONSTRAINT fk_eval_staff FOREIGN KEY (evaluated_by_staff_id) REFERENCES
staff(staff_id) ON DELETE RESTRICT
) ENGINE=InnoDB;

```

```
-- =====
```

```
-- 5) NEW: AUDIT LOGS & NOTIFICATIONS
```

```
-- =====
```

```

CREATE TABLE IF NOT EXISTS audit_logs (
  log_id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  entity_type ENUM('application', 'award', 'disbursement', 'user') NOT NULL,
  entity_id BIGINT UNSIGNED NOT NULL,
  action VARCHAR(100) NOT NULL,
  actor_user_id BIGINT UNSIGNED NULL,
  old_value VARCHAR(255) NULL,

```

```
new_value VARCHAR(255) NULL,  
created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
PRIMARY KEY (log_id),  
KEY idx_audit_entity (entity_type, entity_id)  
) ENGINE=InnoDB;
```

```
CREATE TABLE IF NOT EXISTS notifications (  
notification_id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,  
user_id BIGINT UNSIGNED NOT NULL,  
title VARCHAR(150) NOT NULL,  
message TEXT NOT NULL,  
is_read TINYINT(1) NOT NULL DEFAULT 0,  
is_sent_via_email TINYINT(1) NOT NULL DEFAULT 0,  
created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
PRIMARY KEY (notification_id),  
KEY idx_notifications_user (user_id, is_read),  
CONSTRAINT fk_notifications_user FOREIGN KEY (user_id) REFERENCES users(user_id)  
ON DELETE CASCADE  
) ENGINE=InnoDB;
```

```
-- =====
```

```
-- 6) TRIGGERS
```

```
-- =====
```

```
DELIMITER $$
```

```
DROP TRIGGER IF EXISTS trg_staff_role_faculty$$
```



```
CREATE TRIGGER trg_staff_role_faculty
BEFORE INSERT ON staff
FOR EACH ROW
BEGIN
    DECLARE v_role VARCHAR(20);
    SELECT role INTO v_role FROM users WHERE user_id = NEW.user_id;
    IF v_role IS NULL OR v_role <> 'faculty' THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Faculty profile requires users.role =
faculty';
    END IF;
END$$
```

```
DROP TRIGGER IF EXISTS trg_admins_role_admin$$
CREATE TRIGGER trg_admins_role_admin
BEFORE INSERT ON admins
FOR EACH ROW
BEGIN
    DECLARE v_role VARCHAR(20);
    SELECT role INTO v_role FROM users WHERE user_id = NEW.user_id;
    IF v_role IS NULL OR v_role <> 'admin' THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Admin profile requires users.role =
admin';
    END IF;
END$$
```

```
DROP TRIGGER IF EXISTS trg_students_role_student$$
CREATE TRIGGER trg_students_role_student
```

```

BEFORE INSERT ON students
FOR EACH ROW
BEGIN
    DECLARE v_role VARCHAR(20);
    SELECT role INTO v_role FROM users WHERE user_id = NEW.user_id;
    IF v_role IS NOT NULL AND v_role <> 'student' THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Student profile requires users.role =
student';
    END IF;
END$$

```

```

DELIMITER ;

```

```

-- =====

```

```

-- 7) STORED PROCEDURES

```

```

-- =====

```

```

DELIMITER $$

```

```

DROP PROCEDURE IF EXISTS sp_submit_application$$

```

```

CREATE PROCEDURE sp_submit_application(

```

```

    IN p_application_id BIGINT UNSIGNED,

```

```

    IN p_student_id BIGINT UNSIGNED

```

```

)

```

```

proc: BEGIN

```

```

    DECLARE v_status VARCHAR(20);

```

```

    DECLARE v_open DATETIME;

```

```
DECLARE v_close DATETIME;
```

```
START TRANSACTION;
```

```
SELECT a.status, s.application_open, s.application_close
```

```
    INTO v_status, v_open, v_close
```

```
FROM applications a
```

```
JOIN scholarships s ON s.scholarship_id = a.scholarship_id
```

```
WHERE a.application_id = p_application_id AND a.student_id = p_student_id
```

```
FOR UPDATE;
```

```
IF v_status IS NULL THEN
```

```
    ROLLBACK; SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Application not found';
```

```
END IF;
```

```
IF NOW() < v_open OR NOW() > v_close THEN
```

```
    ROLLBACK; SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Submission window is  
closed';
```

```
END IF;
```

```
IF v_status <> 'draft' THEN
```

```
    ROLLBACK; SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Only draft applications can  
be submitted';
```

```
END IF;
```

```
UPDATE applications SET status = 'submitted', submitted_at = NOW() WHERE  
application_id = p_application_id;
```

```
COMMIT;
```

END\$\$

DROP PROCEDURE IF EXISTS sp_approve_and_award\$\$

CREATE PROCEDURE sp_approve_and_award(

IN p_application_id BIGINT UNSIGNED,

IN p_staff_id BIGINT UNSIGNED,

IN p_amount_awarded DECIMAL(12,2)

)

proc: BEGIN

DECLARE v_scholarship_id BIGINT UNSIGNED;

DECLARE v_term_id BIGINT UNSIGNED;

DECLARE v_student_id BIGINT UNSIGNED;

DECLARE v_user_id BIGINT UNSIGNED;

DECLARE v_staff_user_id BIGINT UNSIGNED;

DECLARE v_max_awards INT;

DECLARE v_current_awards INT;

START TRANSACTION;

-- 1) Safely read AND lock the application first

SELECT scholarship_id, term_id, student_id

INTO v_scholarship_id, v_term_id, v_student_id

FROM applications

WHERE application_id = p_application_id AND status IN ('submitted','under_review')

FOR UPDATE;

```
IF v_scholarship_id IS NULL THEN

    ROLLBACK; SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Application not found or
not approvable';

END IF;
```

-- 2) Lock Scholarship to prevent concurrent over-awarding (Lock order: App -> Scholarship)

```
SELECT max_awards INTO v_max_awards

FROM scholarships WHERE scholarship_id = v_scholarship_id FOR UPDATE;

SELECT COUNT(*) INTO v_current_awards

FROM awards WHERE scholarship_id = v_scholarship_id AND term_id = v_term_id AND
status <> 'cancelled';
```

```
IF v_current_awards >= v_max_awards THEN

    ROLLBACK; SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Scholarship capacity
reached';

END IF;
```

-- 3) Update & Insert

```
UPDATE applications SET status = 'approved' WHERE application_id = p_application_id;

INSERT INTO awards(application_id, scholarship_id, student_id, term_id,
amount_awarded, status)

VALUES (p_application_id, v_scholarship_id, v_student_id, v_term_id,
p_amount_awarded, 'pending');

INSERT INTO reviews(application_id, staff_id, score, decision, comments)
```

```
VALUES (p_application_id, p_staff_id, NULL, 'recommend_approve', 'Approved and  
awarded');
```

```
-- 4) Audit & Notifications
```

```
SELECT user_id INTO v_staff_user_id FROM staff WHERE staff_id = p_staff_id;
```

```
SELECT user_id INTO v_user_id FROM students WHERE student_id = v_student_id;
```

```
INSERT INTO audit_logs (entity_type, entity_id, action, actor_user_id, new_value)
```

```
VALUES ('application', p_application_id, 'STATUS_CHANGED', v_staff_user_id, 'approved');
```

```
IF v_user_id IS NOT NULL THEN
```

```
    INSERT INTO notifications (user_id, title, message)
```

```
    VALUES (v_user_id, 'Application Approved', 'Your scholarship application has been  
approved.');
```

```
END IF;
```

```
COMMIT;
```

```
END$$
```

```
DROP PROCEDURE IF EXISTS sp_pay_disbursement$$
```

```
CREATE PROCEDURE sp_pay_disbursement(
```

```
    IN p_disbursement_id BIGINT UNSIGNED
```

```
)
```

```
proc: BEGIN
```

```
    DECLARE v_award_id BIGINT UNSIGNED;
```

```
    DECLARE v_award_status VARCHAR(20);
```

```
    DECLARE v_awarded DECIMAL(12,2);
```

```
DECLARE v_paid DECIMAL(12,2);
```

```
START TRANSACTION;
```

```
SELECT award_id INTO v_award_id FROM disbursements WHERE disbursement_id =  
p_disbursement_id FOR UPDATE;
```

```
IF v_award_id IS NULL THEN
```

```
    ROLLBACK; SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Disbursement not found';
```

```
END IF;
```

```
SELECT status, amount_awarded INTO v_award_status, v_awarded FROM awards WHERE  
award_id = v_award_id FOR UPDATE;
```

```
IF v_award_status = 'cancelled' THEN
```

```
    ROLLBACK; SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Cannot pay a cancelled  
award';
```

```
END IF;
```

```
UPDATE disbursements SET status = 'paid', paid_at = NOW()
```

```
WHERE disbursement_id = p_disbursement_id AND status = 'scheduled';
```

```
IF ROW_COUNT() > 0 THEN
```

```
    SELECT IFNULL(SUM(amount), 0) INTO v_paid FROM disbursements WHERE award_id =  
v_award_id AND status = 'paid';
```

```
    IF v_paid >= v_awarded THEN
```

```
        UPDATE awards SET status = 'paid' WHERE award_id = v_award_id;
```

```
    ELSE
```

```
    UPDATE awards SET status = 'authorized' WHERE award_id = v_award_id AND status =  
'pending';
```

```
    END IF;
```

```
END IF;
```

```
COMMIT;
```

```
END$$
```

```
DROP PROCEDURE IF EXISTS sp_schedule_semester_disbursements$$
```

```
CREATE PROCEDURE sp_schedule_semester_disbursements(  
    IN p_award_id BIGINT UNSIGNED,  
    IN p_fund_source_id BIGINT UNSIGNED,  
    IN p_eval_sem1_id BIGINT UNSIGNED,  
    IN p_eval_sem2_id BIGINT UNSIGNED  
)  
proc: BEGIN
```

```
    DECLARE v_amount DECIMAL(12,2);
```

```
    START TRANSACTION;
```

```
    SELECT amount_awarded INTO v_amount FROM awards WHERE award_id = p_award_id  
    FOR UPDATE;
```

```
    IF v_amount IS NULL THEN
```

```
        ROLLBACK; SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Award not found';
```

```
    END IF;
```



```

INSERT INTO disbursements(award_id, fund_source_id, amount, status,
evaluation_period_id)

VALUES

(p_award_id, p_fund_source_id, ROUND(v_amount/2, 2), 'scheduled', p_eval_sem1_id),

(p_award_id, p_fund_source_id, v_amount - ROUND(v_amount/2, 2), 'scheduled',
p_eval_sem2_id);

COMMIT;

END$$

```

```

DROP PROCEDURE IF EXISTS sp_record_evaluation_and_release_installment$$

CREATE PROCEDURE sp_record_evaluation_and_release_installment(

IN p_award_id BIGINT UNSIGNED,

IN p_evaluation_period_id BIGINT UNSIGNED,

IN p_staff_id BIGINT UNSIGNED,

IN p_gpa_achieved DECIMAL(5,2),

IN p_notes TEXT

)

proc: BEGIN

DECLARE v_required DECIMAL(5,2);

DECLARE v_scholarship_id BIGINT UNSIGNED;

DECLARE v_decision VARCHAR(10);

DECLARE v_current_standing VARCHAR(20);

START TRANSACTION;

SELECT scholarship_id, standing INTO v_scholarship_id, v_current_standing FROM
awards WHERE award_id = p_award_id FOR UPDATE;

```

IF v_scholarship_id IS NULL THEN

ROLLBACK; SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Award not found';

END IF;

SELECT IFNULL(maintain_min_gpa, 0.00) INTO v_required FROM scholarships WHERE scholarship_id = v_scholarship_id;

SET v_decision = IF(p_gpa_achieved >= v_required, 'pass', 'fail');

INSERT INTO scholar_evaluations(award_id, evaluation_period_id, evaluated_by_staff_id, gpa_achieved, decision, notes)

VALUES (p_award_id, p_evaluation_period_id, p_staff_id, p_gpa_achieved, v_decision, p_notes)

ON DUPLICATE KEY UPDATE evaluated_by_staff_id = VALUES(evaluated_by_staff_id), gpa_achieved = VALUES(gpa_achieved), decision = VALUES(decision), notes = VALUES(notes), evaluated_at = CURRENT_TIMESTAMP;

IF v_decision = 'pass' THEN

UPDATE awards SET standing = 'active' WHERE award_id = p_award_id;

UPDATE disbursements SET status = 'paid', paid_at = NOW() WHERE award_id = p_award_id AND evaluation_period_id = p_evaluation_period_id AND status = 'scheduled';

ELSE

IF v_current_standing = 'active' THEN

UPDATE awards SET standing = 'probation' WHERE award_id = p_award_id;

UPDATE disbursements SET status = 'cancelled' WHERE award_id = p_award_id AND evaluation_period_id = p_evaluation_period_id AND status = 'scheduled';

ELSE

UPDATE awards SET standing = 'terminated', status = 'cancelled' WHERE award_id = p_award_id;

```
UPDATE disbursements SET status = 'cancelled' WHERE award_id = p_award_id AND  
status = 'scheduled';
```

```
END IF;
```

```
END IF;
```

```
COMMIT;
```

```
END$$
```

```
DELIMITER ;
```